Report on Hospital Database

1. Introduction

This report presents the design and implementation of a hospital database system, including the database schema, data generation process, and ethical considerations, to efficiently store and manage information about doctors, departments, patients, and appointments while ensuring data integrity and privacy. The database consists of four tables: **Doctor**, **Department**, **Patient**, and **Appointment**. The data was crafted programmatically using Python libraries such as **Faker**, **NumPy**, **Pandas**, **Random** and **Re**.

2. Data Generation Process

The data was created using Python libraries through a programmatic approach. The following is the summary of the steps involved in the process:

2.1 Tools and Libraries Used

- **Faker**: Used to generate realistic names, email addresses, and dates.
- **NumPy**: Used to develop random numerical data (e.g., ages, salaries).
- **Pandas**: Used to create and manipulate DataFrames.
- **Random**: Used to introduce randomness in data generation (e.g., missing values, duplicates).
- **Re**: Used to produce structured patterns in phone numbers.

2.2 Data Generation Steps

2.2.1. Doctor Table

The generation steps of the Doctor table included creating **50 unique records** for the doctors and then adding **5 missing values** which are denoted by '**nan**' in the Doctor_Name column to replicate real-world data gaps. Then, each doctor was linked to one of the 10 available departments. Additionally, the **email** and **mobile number** of the doctors are **masked** to ensure privacy.

2.2.2. Department Table

Generated ***10 unique*** departments with department names, locations and landline numbers where the landline numbers are masked to ensure data confidentiality.

2.2.3. Patient Table

The stages of creating the patient table comprised of producing 700 unique patient records, followed by the introduction of **15 duplicate** patient names to simulate real-word data inconsistencies. In addition to this, random ages and registration dates are allocated to each patient. The Email and the Mobile_Number of the patient are **masked** to ensure confidentiality.

2.2.4. Appointment Table

The steps involved in constructing an appointment table included the creation of **1300 appointment records** which are connected to doctors, departments and patients by using **foreign keys**.

3. Database Schema

The database schema includes four tables with the following structure that ensures efficient data management, normalization, and data consistency across relationships in the database :-

3.1. Table Relations and Constraints

Tables	Foreign Key	Composite Key	Unique	Not Null	Check	Missing Values	Duplicate	Masked Values
Doctor	Yes					Yes		Yes
Department			Yes					Yes
Patient		Yes		Yes	Yes		Yes	Yes
Appointments	Yes							

3.2.Tables and Columns

3.2.1.Doctor Table

The Doctor table stores information about doctors working in the hospital. This table classifies that each doctor is **uniquely linked** to their respective department by the **foreign key**.

Columns	Data type	Primary key	Unique	Not Null	Check	Foreign Key	Type	Description
Doctor_Id	TEXT	Yes					Nominal	Unique identifier for the table
Doctor_Name	TEXT						Nominal	Name of the doctor
Joining_Date	TEXT						Interval	Date the doctor joined the hospital
Salary	REAL						Ratio	Salary of the doctor
Department	TEXT						Nominal	Department the doctor belongs to. Eg: Cardiology, Neurology.
Department_Id	TEXT					Yes	Nominal	A reference to the Department table, linking doctors to their respective departments. Eg: D006, D009
Email	TEXT						Nominal	Email ID of the doctor
Mobile_Number	TEXT						Nominal	Contact number of the doctor

Doctor Table - Create statement:

```sql
CREATE TABLE "Doctor" (
    "Doctor_Id"  TEXT,
    "Doctor_Name"   TEXT,
    "Joining_Date"   TEXT,
    "Salary" REAL,
    "Department"  TEXT,
    "Department_Id"   TEXT,
    "Email"  TEXT,
    "Mobile_Number"   TEXT,
    PRIMARY KEY("Doctor_Id"),
    FOREIGN KEY("Department_Id") REFERENCES "Department"("Department_Id")
);
```

3.2.2.Department Table

The Department table contains details about the departments of the hospital. This table serves as a reference for the Doctor table, ensuring that each doctor is associated with a valid department and a ***unique constraint*** is added for department column in the department table.

Columns	Data type	Primary key	Unique	Not Null	Check	Foreign Key	Type	Description
Department	TEXT		Yes				Nominal	Unique identifier for each department
Department_Id	TEXT	Yes					Nominal	Name of the department.Eg: Cariology, Neurology.
Location	TEXT						Nominal	Location of the department.Eg:Floor 2, Outpatient Area
Landline_Number	TEXT						Nominal	Landline contact number of the department

Department Table - Create statement:

```sql
CREATE TABLE "Department" (
    "Department"  TEXT UNIQUE,
    "Department_Id"   TEXT,
    "Location" TEXT,
    "Landline_Number"   TEXT,
    PRIMARY KEY("Department_Id")
);
```

3.2.3. Patient Table

The Patient table stores information about patients registered to the hospital. This table details that each patient is uniquely identified while accommodating common data quality issues. Patient_Id and Registration_Date forms the **composite key** of the table and **check constraint** is added to verify that only positive values are inserted into the Age column.

Columns	Data type	Primary key	Unique	Not Null	Check	Foreign Key	Type	Description
Patient_Id	TEXT	Yes					Nominal	Unique identifier for each patient
Patient_Name	TEXT						Nominal	Name of the patient
Age	INTEGER				Age >= 0		Ratio	Age of the patient
Registration_Date	TEXT	Yes					Interval	Date the patient was registered to the hospital
Email	TEXT			Yes			Nominal	Email ID of the patient
Mobile_Number	TEXT			Yes			Nominal	Contact number of the patient

Patient Table - Create statement:

```
CREATE TABLE "Patient" (
    "Patient_Id" TEXT,
    "Patient_Name" TEXT,
    "Age" INTEGER CHECK("Age" >= 0),
    "Registration_Date" TEXT,
    "Email" TEXT NOT NULL,
    "Mobile_Number" TEXT NOT NULL,
    PRIMARY KEY("Patient_Id","Registration_Date")
);
```

3.2.4. Appointment Table

The Appointment table manages details about patient appointments. This table affirms that appointments are linked to both doctors, departments and patients through **foreign keys** while maintaining consistency and integrity.

Columns	Data type	Primary key	Unique	Not Null	Check	Foreign Key	Type	Description
Appointment_Id	TEXT	Yes					Nominal	Unique identifier for each appointment
Patient_Id	TEXT					Yes	Nominal	Reference to the Patient table, linking appointments to patients
Doctor_Id	TEXT					Yes	Nominal	Reference to the Doctor table, linking appointments to doctors
Department_Id	TEXT					Yes	Nominal	Reference to the Department table, linking appointments to departments
Appointment_Date	TEXT						Interval	Date of the appointment
Appointment_Time	TEXT						Interval	Time of the appointment
Appointment_Status	TEXT						Ordinal	Status of the appointment.Eg: Scheduled, Completed, Cancelled

Appointments Table - Create statement:

```sql
CREATE TABLE "Appointments" (
  "Appointment_Id" TEXT,
  "Patient_Id" TEXT,
  "Doctor_Id" TEXT,
  "Department_Id" TEXT,
  "Appointment_Date" TEXT,
  "Appointment_Time" TEXT,
  "Appointment_Status" TEXT,
  PRIMARY KEY("Appointment_Id"),
  FOREIGN KEY("Department_Id") REFERENCES "Department"("Department_Id"),
  FOREIGN KEY("Doctor_Id") REFERENCES "Doctor"("Doctor_Id"),
  FOREIGN KEY("Patient_Id") REFERENCES "Patient"("Patient_Id")
);
```

3.3. Sample Data

3.3.1 **Unique** 10 Departments from Department table

```sql
SELECT *
FROM Department
```

	Department	Department_Id	Location	Landline_Number
1	Cardiology	D001	Floor 2	+1-555-554-77**8
2	Neurology	D002	Wing B, 3rd Floor	+1-555-583-4*4*8
3	Orthopedics	D003	Outpatient Area	+1-555-507-*3*22
4	Pediatrics	D004	Floor 1	+1-555-558-7**17
5	Oncology	D005	Wing D, 4th Floor	+1-555-524-56**0
6	Emergency	D006	Ground Floor	+1-555-561-**771
7	Radiology	D007	Floor 3	+1-555-550-02**6
8	Dermatology	D008	Wing A, 3rd Floor	+1-555-567-*11*9
9	Psychiatry	D009	Floor 5	+1-555-530-*3*60
10	Surgery	D010	Floor 4	+1-555-594-*5*77

3.3.2 Appointment table with **1000+ rows**

```
SELECT COUNT (*) fROM Appointments
```

	COUNT (*)
1	1300

3.3.3 Doctor table with **missing values(denoted by 'nan')** in Doctor Name column and **masked values** in Email and Mobile Number columns

```
SELECT *
FROM Doctor LIMIT 15
```

	Doctor_Id	Doctor_Name	Joining_Date	Salary	Department	Department_Id	Email	Mobile_Number
1	DR0019	nan	2012-03-09	357617.0	Neurology	D002	n**.n**@nhs.com	602-46*-*9*5
2	DR0017	nan	2012-06-05	503285.0	Psychiatry	D009	n**.n**@nhs.com	53*-94*-72*7
3	DR0006	Cassandra Lauren Lin	2015-09-25	444610.0	Psychiatry	D009	c**.l**@nhs.com	836-3**-*349
4	DR0008	Beverly Thomas Mills	2014-07-13	485400.0	Surgery	D010	b**.m**@nhs.com	5**-*21-2598
5	DR0033	Rachael Barbara Nelson	2015-12-25	878024.0	Surgery	D010	r**.n**@nhs.com	299-*4*-6*51
6	DR0009	Jacob Lauren Wilkerson	2004-09-12	868284.0	Orthopedics	D003	j**.w**@nhs.com	972-**1-*028
7	DR0007	Robert Ashlee Waters	2012-01-24	648441.0	Radiology	D007	r**.w**@nhs.com	***-912-2188
8	DR0016	Donald Neil Knight	2000-08-20	175516.0	Surgery	D010	d**.k**@nhs.com	91*-2*2-4*86
9	DR0015	Anna Valerie Johnson	2001-09-28	508808.0	Oncology	D005	a**.j**@nhs.com	6**-515-300*
10	DR0005	Jeffrey Daniel Patel	2013-07-09	519745.0	Surgery	D010	j**.p**@nhs.com	708-*67-42**
11	DR0042	Samuel Luis Downs	2003-03-13	531407.0	Emergency	D006	s**.d**@nhs.com	616-714-8***
12	DR0036	Jessica Brittany Ortiz	2011-01-08	155405.0	Emergency	D006	j**.o**@nhs.com	55*-*31-972*
13	DR0030	Johnathan Samantha Thompson	2015-02-02	396996.0	Dermatology	D008	j**.t**@nhs.com	59*-8**-2101
14	DR0044	nan	2021-12-02	639450.0	Oncology	D005	n**.n**@nhs.com	333-9*1-4*0*
15	DR0001	Chris Stephanie Davis	2008-01-13	425503.0	Oncology	D005	c**.d**@nhs.com	*3*-85*-7188

3.4.4 Patient table with **duplicate** Patient name

```
SELECT *
FROM Patient
WHERE patient_name IN (
  SELECT patient_name
  FROM Patient
  GROUP BY patient_name
  HAVING COUNT(*) > 1
)
ORDER BY patient_name;
```

	Patient_Id	Patient_Name	Age	Registration_Date	Email	Mobile_Number
1	PID0122	Amy Timothy James	5	1959-11-06	a**.j**@gmail.com	**4-640-4*95
2	PID0444	Amy Timothy James	7	1967-05-16	a**.j**@gmail.com	*03-719-41**
3	PID0260	Carrie Michael Logan	12	1980-03-13	c**.l**@gmail.com	8*9-320-7**1
4	PID0457	Carrie Michael Logan	54	1987-11-05	c**.l**@gmail.com	71*-79*-3*46
5	PID0196	Casey Aaron Turner	61	1965-06-15	c**.t**@gmail.com	2*4-*64-*306
6	PID0586	Casey Aaron Turner	14	1952-01-27	c**.t**@gmail.com	**3-261-662*
7	PID0162	Christina Kathy Jordan	37	1986-06-14	c**.j**@gmail.com	6*0-951-0*5*
8	PID0277	Christina Kathy Jordan	22	2015-09-05	c**.j**@gmail.com	763-*48-**49
9	PID0248	Daniel Sherry Guerrero	63	1960-01-12	d**.g**@gmail.com	57*-282-*19*
10	PID0589	Daniel Sherry Guerrero	47	1979-04-18	d**.g**@gmail.com	**5-264-97*1

3.4. Sample code snippets of data generation

3.4.1. Nominal:

```
# Nominal data: Doctors.
doctor_ids = [f'DR{str(i).zfill(4)}' for i in range(1, n+1)]
np.random.shuffle(doctor_ids)

# Select n unique doctor_ids
doctor_ids_data = doctor_ids[:]
print(len(doctor_ids_data))

# Department names and their respective IDs
department_names = ['Cardiology', 'Neurology', 'Orthopedics', 'Pediatrics', 'Oncology',
                    'Emergency', 'Radiology', 'Dermatology', 'Psychiatry', 'Surgery']

print(len(department_names))

# Create a dictionary that maps department names to department IDs
department_id_mapping = {
    'Cardiology': 'D001',
    'Neurology': 'D002',
    'Orthopedics': 'D003',
    'Pediatrics': 'D004',
    'Oncology': 'D005',
    'Emergency': 'D006',
    'Radiology': 'D007',
    'Dermatology': 'D008',
    'Psychiatry': 'D009',
    'Surgery': 'D010'
}
```

3.4.2. Ordinal:

```
# Appointment Statuses
appointment_statuses = ['Scheduled', 'Completed', 'Cancelled']
appointment_status = np.random.choice(appointment_statuses, n, p=[0.7, 0.25, 0.05])
```

3.4.3. Interval:

```
# Generate joining date (Interval Data)
joining_year = np.random.randint(2000, 2024, n)
joining_month = np.random.randint(1, 13, n)
joining_day = np.random.randint(1, 29, n)
joining_date = [f'{joining_year[i]}-{str(joining_month[i]).zfill(2)}-{str(joining_day[i]).zfill(2)}' for i in range(n)]
```

3.4.4. Ratio:

```
# Generate salary data (Ratio Data)
salary_data = np.random.lognormal(mean=13, sigma=0.5, size=n).astype(int)
```

3.4.5. Unique Values:

```
# Create unique patient IDs
patient_ids = [f'PID{str(i).zfill(4)}' for i in range(1, n + 1)]
```

3.4.6. Missing Values:

```
sample_names = [f"{fake.first_name()} {fake.first_name()} {fake.last_name()}" for _ in range(n)]
n_points = 5

sample_names = np.array(sample_names)

# Randomly select 5 indices to set to NaN
random_indices = np.random.choice(len(sample_names), n_points, replace=False)

# Set the selected indices to NaN
sample_names[random_indices] = np.nan
```

3.4.7. Masked Values:

```
# Generate Emails based on first and last names (first_name.last_name@nhs.com)
def generate_masked_email(name):
    first_name, last_name = name.split()[0], name.split()[-1]

    # Mask part of the first name and last name
    masked_first_name = first_name[0] + '*' * 2
    masked_last_name = last_name[0] + '*' * 2

    # Generate the email
    emails = f"{masked_first_name.lower()}.{masked_last_name.lower()}@gmail.com"

    return emails

# Generate masked emails based on sample names
emails = [generate_masked_email(name) for name in patient_names]

emails = np.array(emails)
```

3.4.8. Duplicate values:

```
# Create unique patient IDs
patient_ids = [f'PID{str(i).zfill(4)}' for i in range(1, n + 1)]

# Generate sample names with given, middle, and last names using Faker
patient_names = [f"{fake.first_name()} {fake.first_name()} {fake.last_name()}" for _ in range(n)]

# To generate 15 duplicate names, we'll select 15 unique names and duplicate them
duplicate_count = 15
duplicate_names = np.random.choice(patient_names, duplicate_count, replace=False)

# Replace 15 random names in the original list with the duplicates
replace_indices = np.random.choice(n, duplicate_count, replace=False)
for i, name in zip(replace_indices, duplicate_names):
    patient_names[i] = name

# shuffle the final list to distribute duplicates randomly
np.random.shuffle(patient_names)
```

4 . Justification for Table Design

The **Doctor**, **Department**, **Patient**, and **Appointment** tables were carefully chosen to organize and manage essential healthcare information effectively. The **Doctor table** stores key details about healthcare professionals, including their specialties and contact information, while the **Department table** organizes medical specialties and provides necessary contact details for each department. The **Patient table** keeps track of critical patient information, enabling personalized care and follow-ups, and the **Appointment table** connects patients with doctors, ensuring smooth scheduling and status tracking. Columns like Doctor_Id, Patient_ID, and Department_Id make it easy to identify and categorize individuals, while attributes such as Joining_Date, Salary, Age and Registration_Date offer valuable professional and demographic insights. To simulate real-world challenges, **missing doctor names** and **duplicate patient names** are introduced and sensitive information like Mobille_Number and Email of both doctors and patients, as well as the Landline_Number of the departments are **masked** to comply with privacy regulations. Collectively, these tables and columns create a well-structured, secure, and efficient system that balances functionality, privacy, and compliance, ensuring the hospital runs smoothly and responsibly.

5. Ethical and Data Privacy Considerations

- **Data Accuracy**: Missing values and duplicates were intentionally included to emulate real-world data quality issues, ensuring the database reflects practical scenarios.
- **Data Privacy**: Sensitive details like email addresses, contact numbers, doctor names and patient names were incorporated using the Faker library to secure no real personal data was used.

6. Conclusion

The hospital database is designed to effectively organise and manage information about doctors, departments, patients, and appointment. The database assures data integrity and scalability by incorporating normalization, foreign keys, and appropriate data types. Privacy and ethical concerns were prioritized to establish data protection and safeguard sensitive information.